

TRABALHO PRÁTICO N.º2

Comunicações por Computador

Universidade do Minho
Licenciatura em Engenharia Informática

Realizado por:
Marco Sèvegrand, A106907
Francisco Martins, A106902
Hugo Soares, A107293

ÍNDICE

1. Introdução
2. Arquitetura do Sistema e Topologia
 - 2.1 Segmento Terrestre
 - 2.2 Segmento Orbital
 - 2.3 Segmento de Superfície
3. Desenho dos Protocolos
 - 3.1 Serialização e Codificação
 - 3.2 Protocolo MissionLink (RUDP)
 - 3.3 Protocolo TelemetryStream (TCP)
4. Implementação dos Componentes
 - 4.1 Simulação do Rover
 - 4.2 Nave-Mãe e Gestão de Estado
 - 4.3 API de Observação e Ground Control
5. Testes e Validação
 - 5.1 Perda de Pacotes
 - 5.2 Latência
 - 5.4 Crash & Recovery
 - 5.5 Múltiplos Rovers
6. Utilização de Inteligência Artificial
7. Análise Crítica e Conclusão
 - 7.1 Robustez
 - 7.2 Limitações e Melhorias Futuras
 - 7.3 Conclusão
8. Apêndice
 - 8.1 Guia de Utilização
 - 8.2 Anexos

1. INTRODUÇÃO

O presente projeto tem como objetivo principal desenhar, implementar e validar uma arquitetura de comunicação distribuída robusta, capaz de suportar os desafios complexos de uma missão espacial simulada. Num cenário onde a fiabilidade e a eficiência são críticas, o sistema recria as condições de operação entre uma Nave-Mãe em órbita e uma frota de Rovers autônomos na superfície de um planeta. A comunicação neste ambiente é bidirecional e constante, com a Nave-Mãe a atuar como um coordenador central responsável por distribuir missões geográficas, enquanto os Rovers reportam a sua telemetria e o estado das tarefas em tempo real. Para aproximar a simulação da realidade, a infraestrutura de rede não liga apenas a origem ao destino, mas inclui uma malha de satélites que atuam como intermediários de encaminhamento, introduzindo constrangimentos típicos do espaço como latência elevada, variação no atraso de entrega e perda de pacotes.

Para mitigar estes desafios e garantir a integridade das operações, a arquitetura proposta assenta no desenvolvimento de dois protocolos de transporte distintos. O primeiro, denominado MissionLink, é implementado sobre UDP e destina-se à transmissão de dados críticos, como o envio de missões. Como o UDP não garante a entrega por natureza, foram implementados mecanismos de fiabilidade diretamente na camada de aplicação, incluindo confirmações de receção, retransmissão automática, sequenciação de mensagens e gestão de sessões. Um ponto de destaque nesta implementação é a utilização de Correção de Erros Direta, ou FEC, que se revela essencial para evitar a espera por retransmissões num cenário onde a latência é muito custosa.

Paralelamente, para assegurar o envio de grandes volumes de dados sensoriais dos rovers para a Nave-Mãe, foi desenhado o protocolo TelemetryStream sobre TCP. A escolha deste protocolo visa garantir a integridade da informação monitorizada, focando-se a implementação do lado do cliente em estratégias de resiliência, como a reconexão automática com tempos de espera progressivos para recuperar o fluxo de dados após falhas temporárias na rede. Os próprios Rovers simulam comportamentos físicos reais, como o consumo de bateria, o movimento e a recolha de dados ambientais, enquanto a Nave-Mãe processa toda esta informação de forma concorrente.

Para completar o ecossistema, o projeto integra um módulo de Ground Control que expõe o estado do sistema ao exterior. Através de uma interface gráfica alimentada por uma API de Observação, os operadores humanos podem visualizar a posição e o estado da frota em tempo real, bem como submeter novas missões. O trabalho finaliza com a validação de toda esta arquitetura em cenários adversos, colocando à prova a capacidade do sistema de manter a operacionalidade e a consistência dos dados mesmo perante condições de rede severamente degradadas.

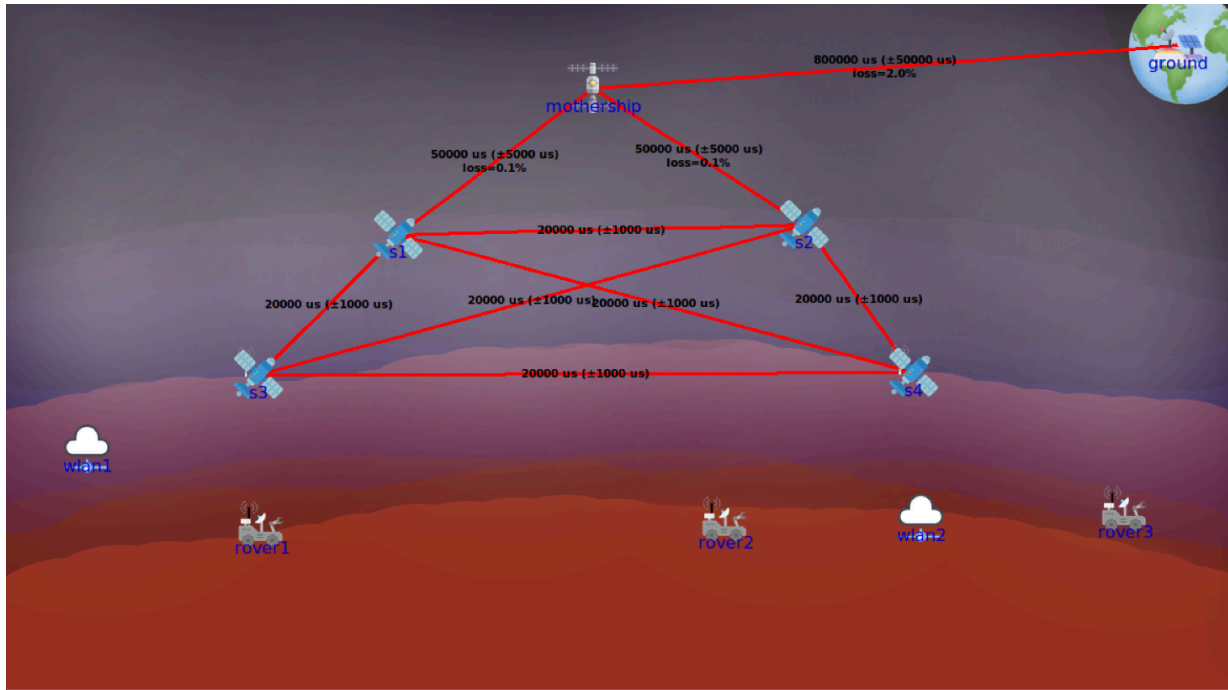
2. ARQUITETURA DO SISTEMA E TOPOLOGIA

A infraestrutura de comunicação foi modelada utilizando a ferramenta CORE, configurada para replicar a hierarquia de uma rede espacial. A topologia divide-se em três camadas lógicas: Terrestre, Orbital e Superfície. Todas estão interligadas por endereçamento IPv4 privado (10.0.x.x/24).

Importa notar uma decisão de design na emulação: embora ligações físicas Ethernet não sejam viáveis no vácuo espacial, utilizámos esta norma como uma abstração da Camada de Ligação de Dados (L2). Esta opção permite focar o estudo e a validação nas camadas superiores (Rede, Transporte e Aplicação). Assim, isolamos a

complexidade dos drivers de rádio ou links óticos, assumindo que o enlace físico está garantido.

Encaminhamento Dinâmico (OSPF) A gestão de rotas é assegurada dinamicamente pelo protocolo OSPF (Open Shortest Path First). Este serviço (v2 e v3) encontra-se ativo exclusivamente no backbone da rede, ou seja, na Nave-Mãe e nos Satélites. Desta forma, a infraestrutura orbital consegue reconfigurar rotas automaticamente em caso de falha. Já os Rovers, sendo nós terminais com recursos limitados, não participam na descoberta de rotas e comunicam apenas com o seu gateway imediato.



2.1. Segmento Terrestre (Ground Segment)

2.1.1 Nó Ground (ID 1)

Representa o Centro de Controlo de Missão na Terra. Este nó aloja a aplicação Ground Control e conecta-se à rede espacial através de um link ponto-a-ponto dedicado com a Nave-Mãe, simulando o uplink/downlink principal (sub-rede 10.0.0.0/24).

2.2. Segmento Orbital (Space Segment)

2.2.1 Nave-Mãe (Mothership, ID 2)

É o nó central, atuando simultaneamente como gateway de comunicações e orquestrador da missão. Possui interfaces distintas para comunicar com a Terra e para se ligar à constelação de satélites (conectando-se diretamente a s1 e s2).

2.2.2 Constelação de Satélites (s1 a s4)

Estes quatro nós (IDs 3 a 6) formam o backbone de comunicação. Atuam como routers transparentes na camada de rede e são vitais para o transporte de dados, embora não executem a lógica aplicacional da missão. Estão organizados numa topologia de malha completa (full mesh), onde cada satélite tem ligação direta a todos os outros. Esta configuração maximiza a redundância, garantindo que existem sempre caminhos alternativos para o encaminhamento de dados caso um link falhe.

2.3. Segmento de Superfície (Surface Segment)

2.3.1 Redes de Acesso (WLANS)

A cobertura na superfície é garantida por duas redes sem fios, configuradas com 54 Mbps de largura de banda e uma latência artificial de 5ms para simular o atraso de propagação rádio.

wlan1 (ID 7): Servida pelo satélite s3.

wlan2 (ID 8): Servida pelo satélite s4.

2.3.2 Rovers

Os veículos de exploração operam como nós terminais nestas redes:

rover1 (ID 9): Está na zona de cobertura da wlan1.

rover2 (ID 10) e rover3 (ID 11): Partilham a wlan2, competindo pelos mesmos recursos de rede para enviar a sua telemetria.

3. DESENHO DOS PROTOCOLOS

3.1 Serialização e Codificação

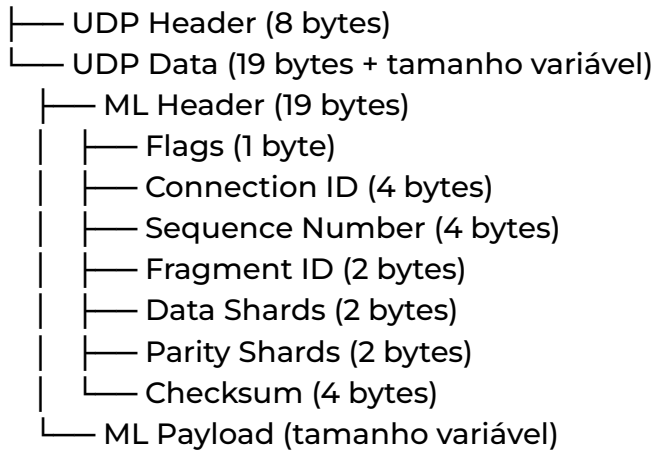
Enquanto a comunicação externa com o Ground Control utiliza JSON sobre HTTP, a rede interna da missão aposta numa codificação binária feita à medida. A decisão de abandonar formatos de texto, como JSON ou XML, teve um objetivo claro: reduzir drasticamente o tamanho dos pacotes (overhead) e acelerar o processamento em cada nó da rede. Todo o processo de serialização é centralizado no pacote codecs, uma camada desenhada para ser agnóstica ao protocolo de transporte. Isto separa a lógica de apresentação da lógica de envio, permitindo reutilizar os mesmos mecanismos de forma transparente, quer o canal subjacente seja UDP (no MissionLink) ou TCP (no TelemetryStream).

A conversão dos dados respeita sempre a ordem Big-Endian, mas a estratégia adapta-se à natureza da informação. Nas Mensagens de Missão, implementa-se uma lógica semelhante a Type-Length-Value (TLV) para suportar o polimorfismo: o pacote começa com um cabeçalho de 1 byte que identifica o tipo de mensagem (0x01, 0x02 ou 0x03), indicando ao decodificador qual a estrutura a instanciar antes de ler o resto da sequência. Já na Telemetria, os dados são serializados num bloco contínuo onde a eficiência é prioridade. Estados ou opções finitas, que ocupariam vários bytes como texto, são convertidos em inteiros de 8 bits (uint8), e estruturas de geometria variável, como a GeographicArea, recebem um tratamento específico onde a definição da forma precede as coordenadas.

3.2 Protocolo MissionLink (RUDP)

O MissionLink é o protocolo crítico do sistema, sendo o grande responsável pela gestão e atribuição de missões. Como a arquitetura assenta no protocolo de transporte UDP, foi necessário desenvolver uma camada de fiabilidade aplicacional (RUDP), designada por udplink. Esta camada implementa mecanismos robustos para garantir que a comunicação mantém a sua integridade e eficiência, mesmo num ambiente espacial adverso.

UDP Datagram



3.2.1. Fragmentação

O protocolo implementa fragmentação ao nível da aplicação para lidar com volumes de dados que excedem a Unidade Máxima de Transmissão (MTU) da rede. Na prática, os dados serializados são divididos em blocos menores, chamados shards, cujo tamanho é calculado dinamicamente com base no MTU configurado. Cada um destes fragmentos transporta um cabeçalho de 19 bytes com informação vital para a remontagem, incluindo o índice do fragmento e os totais de fragmentos de dados e de paridade.

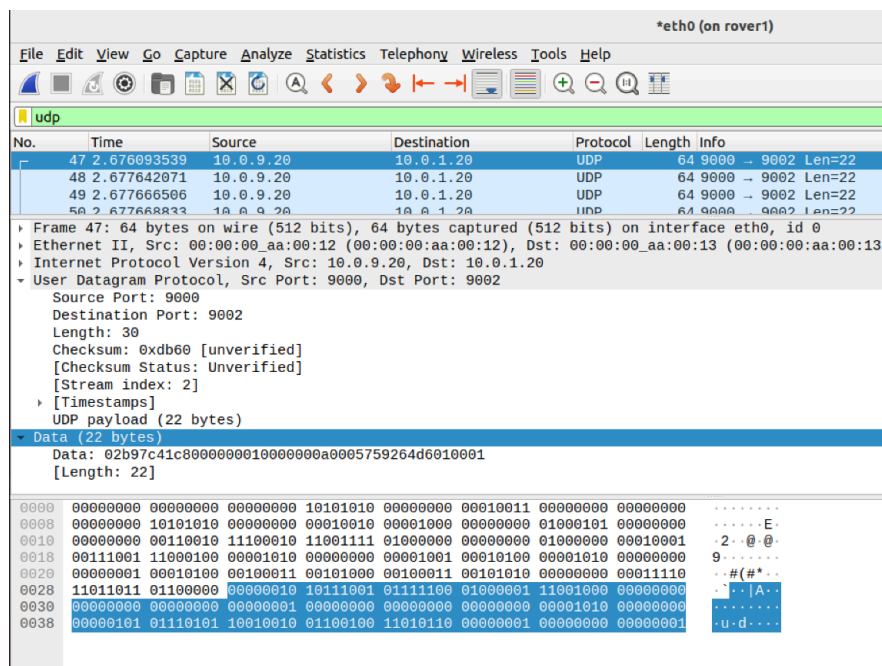
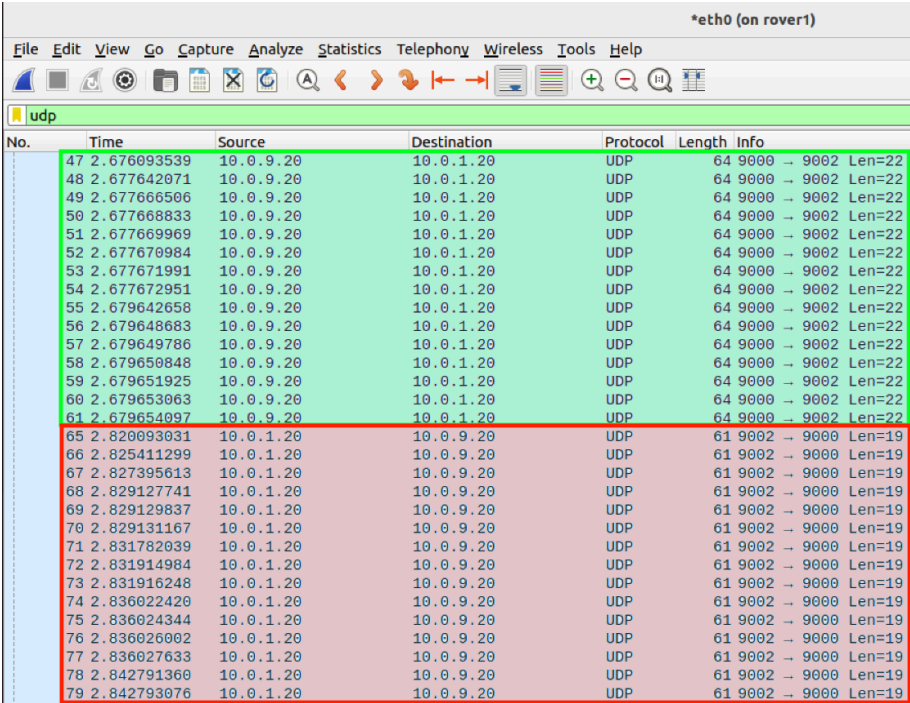


Figura 1: Captura de um pacote UDP (fragmento) no Wireshark

A Figura 1 valida a implementação desta estrutura. A captura evidencia um pacote UDP com um payload total de 22 bytes. Sabendo que o cabeçalho do protocolo udplink ocupa fixamente 19 bytes, deduz-se que este fragmento transporta exatamente 3 bytes de dados úteis (payload efetivo). Este comportamento é consistente com a lógica de fragmentação configurada, onde o sistema divide a mensagem em pequenos blocos para permitir a aplicação de redundância e evitar fragmentação ao nível do IP. O campo "Data" visível na captura confirma a presença do cabeçalho binário seguido da curta sequência de dados.

3.2.2. Confirmações (Acknowledgements)

A fiabilidade é garantida através de um diálogo constante entre as partes. O recetor envia um pacote de confirmação (ACK) por cada fragmento de dados que recebe com sucesso. Do lado de quem envia, o sistema mantém um registo detalhado do estado de cada pacote num vetor de booleanos, permitindo saber exatamente que partes da mensagem chegaram ao destino e quais ainda estão em falta.



No.	Time	Source	Destination	Protocol	Length	Info
47	2.676093539	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
48	2.677642071	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
49	2.677666506	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
50	2.677668833	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
51	2.677669969	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
52	2.677670984	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
53	2.677671991	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
54	2.677672951	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
55	2.679642658	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
56	2.679648683	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
57	2.679649786	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
58	2.679650848	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
59	2.679651925	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
60	2.679653063	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
61	2.679654097	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002 Len=22
65	2.820093031	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
66	2.825411299	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
67	2.827395613	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
68	2.829127741	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
69	2.829129837	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
70	2.829131167	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
71	2.831782039	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
72	2.831914984	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
73	2.831916248	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
74	2.836022420	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
75	2.836024344	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
76	2.836026002	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
77	2.836027633	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
78	2.842791360	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19
79	2.842793076	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000 Len=19

Figura 2: Captura de uma mensagem (fragmentos + ACKs) no Wireshark

Na Figura 2, observa-se o fluxo completo de uma transação MissionRequest entre o Rover 1 (10.0.9.20) e a Nave-mãe (10.0.1.20). Na Zona Verde (Dados), identificam-se múltiplos pacotes de 22 bytes enviados pelo Rover (porta 9000) para a Nave-mãe (porta 9002). Estes correspondem aos fragmentos da mensagem descritos na secção anterior. Na Zona Vermelha (ACKs), em resposta, a Nave-mãe envia pacotes de 19 bytes para o Rover. O tamanho de 19 bytes é tecnicamente relevante, pois corresponde exatamente ao tamanho do cabeçalho do protocolo sem qualquer payload adicional, confirmando que se tratam de pacotes de controlo (ACKs) puros.

3.2.3. Retransmissões

Para combater a perda de pacotes sem congestionar a rede, foi implementado um sistema de retransmissão inteligente. Em vez de reenviar todo o pacote, aplica-se uma retransmissão seletiva apenas dos fragmentos que não receberam confirmação. Além disso, o intervalo entre tentativas aumenta exponencialmente a cada falha, prevenindo o colapso da comunicação em situações de congestionamento severo ou falhas temporárias.

3.2.4. Deteção de Erros (Error Detection)

A integridade dos dados é validada antes de qualquer processamento lógico. Cada fragmento inclui um campo Checksum CRC32 calculado sobre o cabeçalho e o conteúdo. Assim que um fragmento chega, o destinatário recalcula este valor e compara-o. Se não corresponder, o fragmento é imediatamente descartado como corrompido, o que acabará por forçar uma retransmissão visto que nenhum ACK será enviado.

3.2.5. Forward Error Correction (FEC)

Em cenários de latência elevada, típicos do espaço, o custo temporal de esperar por uma retransmissão pode ser proibitivo. Para mitigar este problema, integrámos Forward Error Correction utilizando a codificação Reed-Solomon. O emissor gera fragmentos de paridade adicionais, permitindo ao recetor reconstruir a mensagem original assim que receber um número suficiente de fragmentos, recuperando dados perdidos sem nunca ter de solicitar um reenvio à origem.

3.2.6. Gestão de Concorrência

Dado o volume de tráfego e a natureza assíncrona da rede, o sistema implementa um controlo de concorrência rigoroso para evitar a exaustão de recursos. Foram criadas pools de goroutines distintas para processar pacotes e entregar mensagens, atuando como semáforos que limitam o número de tarefas simultâneas. Se a capacidade máxima for atingida, o excesso é descartado para proteger a estabilidade do nó.

3.2.7. Entrega Ordenada (In-Order Delivery)

Para garantir a consistência da missão, o protocolo assegura que as mensagens são entregues à aplicação na ordem exata de envio, utilizando números de sequência e IDs de conexão. Se um pacote chegar fora de ordem, fica retido em buffer até que a sequência esteja completa. Para evitar bloqueios indefinidos, um temporizador de lacuna monitoriza o processo e avança se o pacote em falta demorar demasiado.

3.2.8. Abstração e Reutilização

A implementação no pacote `udplink` tira partido dos Genéricos do Go para tornar a camada de transporte totalmente independente do tipo de dados. Embora neste projeto o protocolo seja usado para transportar mensagens de missão, a sua arquitetura é agnóstica e pode ser reutilizada sem alterações para transmitir qualquer outra estrutura de dados de forma fiável, bastando fornecer os codificadores apropriados.

3.3 Protocolo TelemetryStream (TCP)

O TelemetryStream assume o papel de canal dedicado à monitorização contínua da frota. Ao contrário do MissionLink, que gere eventos e comandos pontuais, este protocolo orienta-se a fluxos de dados, transportando atualizações periódicas de estado da superfície para a órbita. A implementação baseia-se no pacote `tcpstream` e tira partido das garantias nativas do TCP, como fiabilidade, controlo de fluxo e entrega ordenada, adicionando depois a lógica necessária de delimitação e resiliência aplicacional.

3.3.1. Delimitação de Mensagens (Framing)

Como o TCP funciona como um fluxo contínuo de bytes e não distingue mensagens discretas, não preserva nativamente as fronteiras dos pacotes da aplicação. Para resolver este desafio, o protocolo implementa um mecanismo de prefixo de tamanho (Length-Prefixing). Na prática, cada mensagem de telemetria recebe um cabeçalho de 4 bytes, codificado como inteiro unsigned em Big-Endian, que indica o tamanho exato do conteúdo. O servidor utiliza este prefixo para saber quantos bytes deve ler a seguir, garantindo que cada estrutura é isolada e reconstruída corretamente antes de ser entregue ao decodificador, o que previne erros de leitura no fluxo.

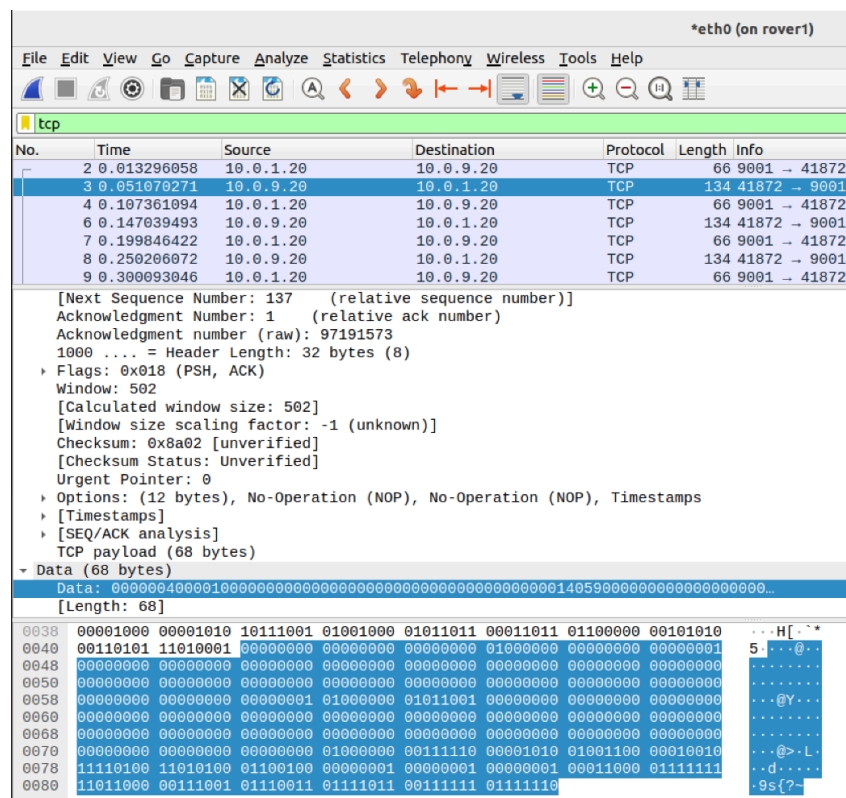


Figura 3: Captura de um pacote TCP no Wireshark

A Figura 3 valida a estrutura do pacote na rede. A captura destaca um segmento enviado pelo Rover 1 (10.0.9.20) para a Nave-mãe (10.0.1.20) na porta 9001. O painel de inspeção confirma um payload TCP total de 68 bytes, corroborando o dimensionamento teórico do protocolo. É possível isolar no início dos dados (ver destaque no dump hexadecimal) a sequência de 4 bytes 00 00 00 40. Este valor corresponde ao cabeçalho de tamanho (uint32 Big Endian), onde 0x40 hexadecimal equivale a 64 decimal. Isto instrui o decodificador de que se seguem exatamente 64 bytes de dados de telemetria serializados, perfazendo o total de 68 bytes (4 bytes de cabeçalho + 64 bytes de estrutura de dados) transmitidos por cada atualização de estado.

3.3.2. Resiliência e Reconexão Automática

Dada a instabilidade natural da topologia espacial, o cliente no Rover foi desenhado com mecanismos de tolerância a falhas para manter o fluxo de dados. O sistema mantém um ciclo de envio contínuo que recolhe dados dos sensores e os transmite na frequência definida. Se a ligação cair, seja por falha de rede ou reinício da Nave-Mãe, o cliente deteta a desconexão e inicia a recuperação. Utiliza-se aqui um algoritmo de Backoff Exponencial que espaça progressivamente as tentativas de reconexão, evitando sobrecarregar a rede com pedidos simultâneos e permitindo que a estabilidade seja recuperada de forma orgânica.

3.3.3. Gestão de Concorrência no Servidor

Do lado da Nave-Mãe, o sistema atua como um servidor TCP concorrente, preparado para agregar e processar fluxos de vários rovers ao mesmo tempo. Para cada nova conexão aceite, é lançada uma goroutine dedicada. Esta estratégia isola totalmente o processamento de cada veículo, assegurando que um cliente mais lento ou bloqueado não interfere na receção de dados do resto da frota.

3.3.4. Abstração e Reutilização

Tal como acontece no MissionLink, a implementação do TelemetryStream recorre aos Genéricos do Go para garantir flexibilidade. Embora neste projeto o módulo seja usado para transportar a estrutura de telemetria, o código de transporte é agnóstico ao tipo de dados. O sistema depende apenas das funções de codificação e decodificação fornecidas no arranque, permitindo reutilizar este módulo para qualquer outro fluxo contínuo na arquitetura sem ser preciso alterar o código base.

4. IMPLEMENTAÇÃO DOS COMPONENTES

4.1 Simulação do Rover

O componente Rover funciona como o elo de ligação entre a lógica de controlo autónomo e a simulação física num ambiente distribuído. A sua arquitetura foi pensada para garantir um desacoplamento claro entre o que chamamos "hardware virtual" e a camada de rede, colocando o ComputeElement no centro de tudo. Esta estrutura atua como o computador de bordo, mantendo o estado interno do veículo e reagindo à passagem do tempo, enquanto os clientes de rede TCP e UDP funcionam como interfaces periféricas que consultam ou alteram este estado de forma concorrente, sem nunca bloquear o processamento central.

4.1.1. Gestão de Estado e Concorrência

Para garantir a integridade dos dados num ambiente onde vários processos correm ao mesmo tempo, segmentámos o estado em contentores genéricos `safe.Var[T]`. Estes contentores encapsulam as primitivas de sincronização necessárias, como `sync.RWMutex`. Na prática, organizamos as variáveis em três grupos: as variáveis de estado, que monitorizam a saúde dos subsistemas, a bateria e o estado operacional do rover; as variáveis espaciais, que guardam a posição e velocidade; e as variáveis de missão, que armazenam o contexto da tarefa e o buffer de dados. Esta abordagem permite que operações de leitura e escrita aconteçam simultaneamente sem risco de corrupção de memória.

4.1.2. Ciclo de Computação e Simulação Física

O ciclo principal de simulação (`computeLoop`) é executado a uma frequência fixa de 20Hz (50ms). A cada iteração, o sistema percorre os módulos de simulação para atualizar as propriedades físicas com base no tempo que passou. O Módulo de Energia aplica um modelo de descarga dinâmico, onde o consumo aumenta ou diminui consoante o esforço do rover, simulando ainda o recarregamento passivo. Já o Módulo de Motores calcula a velocidade e a aceleração, enquanto o Módulo de Sensores traduz esse movimento em novas coordenadas cartesianas. Para resolver os erros de precisão típicos da vírgula flutuante, implementámos um mecanismo de "snapping" espacial que arredonda a posição do rover quando este está muito perto de coordenadas inteiras, garantindo que chega com precisão ao destino.

4.1.3. Lógica de Missão e Navegação

A execução de missões tem o seu próprio ritmo, gerido por um ciclo dedicado que acorda assim que chega um `MissionAssignment`. O planeamento da trajetória utiliza o pacote `geo` para transformar a área alvo em pontos discretos, que são depois ordenados pelo algoritmo `Nearest Neighbor` para garantir que o rover faz o caminho mais curto. Durante a viagem, o veículo monitoriza ativamente a sua própria condição. Se o tempo limite for excedido ou se a saúde dos sistemas degradar demasiado, a missão é abortada por segurança. A recolha de dados, como amostras ou imagens, é simulada através da geração de payloads sintéticos que vão sendo acumulados num buffer de saída.

4.1.4. Subsistema de Rede e Telemetria

A comunicação com o exterior acontece em paralelo, garantindo que o processamento de rede nunca bloqueia a simulação física. No arranque, o Rover inicializa o TelemetryStream sobre TCP, estabelecendo uma conexão persistente com a Nave-Mãe. Uma goroutine dedicada recolhe periodicamente o estado do ComputeElement, consolidando a informação de todos os módulos num objeto de telemetria que é enviado para a órbita. Ao mesmo tempo, o protocolo MissionLink sobre UDP gere dois ciclos vitais: um que pede trabalho quando o rover está livre e outro que envia os dados da missão. Este último inclui um mecanismo de keep-alive que envia atualizações vazias na ausência de dados, evitando que a Nave-Mãe assuma erradamente que a missão parou por falta de comunicação.

4.2 Nave-Mãe e Gestão de Estado

A Nave-Mãe funciona como o cérebro de toda a operação, sendo a responsável por manter a coerência do estado global e coordenar a frota. A sua implementação reflete uma arquitetura focada na disponibilidade e no processamento concorrente. O servidor consegue gerir, em simultâneo, fluxos de telemetria de alta frequência via TCP, mensagens de controlo crítico via UDP e uma API via HTTP. Para suportar esta carga heterogénea sem criar latência ou condições de corrida, o núcleo da aplicação evita bloqueios globais que parariam todo o sistema. Em vez disso, optámos por uma gestão de estado baseada em estruturas de dados concorrentes e bloqueios granulares aplicados apenas onde são estritamente necessários.

4.2.1. Arquitetura de Dados e Gestão de Estado

Gerir o estado em memória é o alicerce desta arquitetura. Para minimizar conflitos de escrita, segmentámos o modelo de dados em dois domínios principais. O primeiro gere a frota através do mapa roverTelemetry, que associa os identificadores dos rovers a contentores seguros do tipo safe.Var. Esta organização é fundamental, pois permite ao subsistema de rede atualizar a posição de um veículo específico bloqueando apenas esse rover, deixando os outros processos livres para ler ou escrever dados dos restantes. Utilizamos também mapas auxiliares para rastrear rapidamente a vivacidade das conexões e a disponibilidade dos rovers, evitando leituras pesadas desnecessárias. O segundo domínio foca-se nas missões, utilizando um repositório central e uma lista duplamente ligada que funciona como uma fila de prioridade para tarefas pendentes. Isto garante operações de inserção e remoção eficientes, essenciais para o desempenho do algoritmo de escalonamento.

4.2.2. Lógica de Decisão e Atribuição de Missões

A parte mais complexa da lógica reside no método assignMission, que é ativado sempre que um rover pede trabalho via UDP. Antes de atribuir qualquer tarefa, a Nave-Mãe faz uma avaliação rigorosa. Primeiro, verifica atomicamente o estado do rover; se este estiver em erro ou marcado como desconhecido, o pedido é logo descartado para não comprometer a missão. Se o rover estiver operacional e livre, entra em ação um algoritmo "guloso". O sistema percorre as missões pendentes, calcula a distância euclidiana até à posição do rover e escolhe a mais próxima. A atribuição é feita numa transação atômica que remove a tarefa da lista e a vincula ao rover, garantindo que nunca existem dois veículos com a mesma missão, mesmo sob forte concorrência.

4.2.3. Mecanismos de Monitorização e Recuperação de Falhas

Para garantir a resiliência numa rede propensa a falhas, a Nave-Mãe implementa um monitor proativo chamado Watchdog, que corre num ciclo independente a cada 50 milissegundos. Este componente verifica a "frescura" dos dados recebidos. Se um

rover ficar em silêncio durante demasiado tempo, o sistema força o seu estado para "Desconhecido". Isto alerta o Ground Control sem remover o veículo do sistema, permitindo uma eventual recuperação. Mais crítica é a monitorização das missões ativas: se uma tarefa deixar de receber atualizações de progresso, assumimos que o rover falhou. Para evitar bloqueios perpétuos de recursos, a missão é marcada como estagnada e o rover é desvinculado. Esta limpeza é vital, pois liberta o veículo no sistema para que, caso recupere a comunicação, possa receber novas ordens ou retomar o trabalho.

4.2.4. Interface de Observação e Integração Externa

Por fim, a Nave-Mãe abre uma janela para o exterior através de uma API de Observação. Este servidor HTTP dedicado alimenta a interface Ground Control e permite a injeção dinâmica de novas tarefas. A segurança dos dados é garantida por um padrão de isolamento estrito: as respostas JSON são construídas a partir de cópias instantâneas (snapshots) do estado atual. Esta abordagem desacopla completamente as operações de leitura da API das escritas críticas de telemetria. O resultado é que a visualização do estado em tempo real nunca afeta o desempenho ou a estabilidade da coordenação da missão, mesmo que múltiplos operadores estejam a consultar o sistema simultaneamente.

4.3 Ground Control

O Ground Control é a interface visual do sistema, funcionando como uma Single Page Application (SPA) que nos permite ver e controlar toda a operação. A sua arquitetura divide-se em duas partes: um servidor em Go, que trata de entregar o site ao navegador, e uma aplicação em React, que trata de tudo o que vemos no ecrã e da comunicação com a API.

4.3.1. Servidor de Aplicação e Configuração Dinâmica

A entrega do frontend é feita por um servidor HTTP próprio. Este servidor resolve de forma prática um problema comum: como configurar uma aplicação estática num ambiente que pode mudar. A solução passa por criar um script de configuração no momento em que a página abre. Esse script injeta o endereço da Nave-Mãe diretamente no navegador. A grande vantagem disto é que podemos usar exatamente o mesmo programa em qualquer rede sem ter de recompilar nada. A interface sabe sempre onde se ligar e o servidor limita-se a entregar os ficheiros de forma eficiente.

4.3.2. Sincronização de Dados e Gestão de Estado

A lógica principal do cliente está no componente App.tsx, que usa uma estratégia de polling para manter os dados atualizados. Basicamente, a aplicação está constantemente a pedir informações à Nave-Mãe em intervalos regulares. Para evitar problemas se a rede estiver lenta, existe um mecanismo de segurança que impede novos pedidos de saírem enquanto o anterior não terminar. Como o estado de toda a frota é gerido num único sítio, a interface consegue calcular métricas em tempo real e aplicar filtros nos dados, isolando os rovers por estado ou pelo progresso da missão.

5. TESTES E VALIDAÇÃO

A validação da arquitetura foi realizada num ambiente controlado utilizando a ferramenta CORE. Este ambiente permitiu-nos replicar a topologia da rede espacial descrita anteriormente e, mais importante, introduzir perturbações controladas nas ligações, como latência, perda de pacotes e jitter. O objetivo foi colocar à prova a robustez dos protocolos MissionLink e TelemetryStream. A estratégia de testes focou-se em cinco vetores críticos de falha.

5.1 Perda de Pacotes (Packet Loss)

Neste cenário, configurámos uma taxa de perda de pacotes sintética nas interfaces dos satélites, variando entre 5% e 30%, para testar a fiabilidade da camada aplicacional do MissionLink. O protocolo UDP, por natureza, descartou os segmentos perdidos, mas a nossa camada de fiabilidade no udplink detetou estas falhas através da ausência de confirmações (ACKs). O sistema de retransmissão seletiva reagiu como esperado, reenviando apenas os fragmentos em falta em vez de repetir a mensagem toda, o que otimizou a largura de banda. Em situações de perda moderada, o mecanismo de correção de erros (FEC) brilhou ao permitir reconstruir a mensagem original usando apenas os fragmentos de paridade, sem sequer pedir retransmissão. Isto prova que o protocolo consegue manter a integridade das missões mesmo quando a rede falha significativamente.

5.2 Latência e Atraso de Propagação

Para simular a distância real da órbita, introduzimos uma latência de 5000ms nas ligações. No TelemetryStream sobre TCP, o protocolo ajustou a janela de transmissão e manteve a entrega ordenada, resultando num atraso visual no Ground Control que é consistente com a física do cenário. Já no MissionLink sobre UDP, o desafio foi a calibração. Com os timeouts padrão de 3 segundos, o sistema assumia erradamente que os pacotes se tinham perdido. O problema ficou resolvido ao ajustar estes parâmetros para valores superiores ao tempo de ida e volta (RTT) e ao aplicar um backoff exponencial, prevenindo o congestionamento da rede com reenvios desnecessários. O mecanismo de watchdog também funcionou corretamente: quando o atraso impedia a chegada dos sinais de vida a tempo, os rovers passavam temporariamente a "Desconhecido" e recuperavam automaticamente assim que os dados chegavam.

5.3 Recuperação de Falhas (Crash & Recovery)

Este teste avaliou a capacidade de recuperação perante falhas críticas. Primeiro, forçámos a paragem abrupta e o reinício de um Rover. A Nave-Mãe detetou a nova sessão graças ao identificador de conexão (CID) no cabeçalho, distinguindo os pacotes da sessão antiga, que foram descartados, dos novos, permitindo uma resincronização imediata. De seguida, cortámos a ligação de rede durante o envio de telemetria. O cliente no Rover detetou o erro de escrita e ativou a reconexão automática, realizando tentativas cada vez mais espaçadas (1s, 2s, 4s...) até restabelecer a ligação com sucesso e retomar o fluxo sem intervenção humana.

5.4 Múltiplos Rovers (Concorrência)

Para validar a robustez do servidor sob carga, decidimos escalar o teste para lá dos limites da topologia visualizada no CORE, que possui apenas três nós de superfície. O teste de concorrência envolveu a operação simultânea de seis rovers. Para concretizar este cenário sem alterar o desenho da rede, instanciámos os processos adicionais a correr localmente (localhost) mas configurados com portas distintas. Esta abordagem permitiu-nos simular uma frota alargada a competir ativamente por recursos e validação de missões, colocando uma pressão real sobre a capacidade de gestão da Nave-Mãe.

A utilização intensiva de estruturas de dados concorrentes e variáveis seguras na Nave-Mãe garantiu a total ausência de condições de corrida (Race Conditions). Mesmo com seis clientes a bombardear o servidor com pedidos simultâneos, a API de Observação continuou a servir dados consistentes ao Ground Control, refletindo o estado de cada uma das instâncias sem corrupção de memória. O algoritmo de

distribuição de missões validou-se eficazmente neste cenário de maior densidade. Com várias missões disponíveis e múltiplos rovers em estado de espera, a Nave-Mãe atribuiu corretamente cada tarefa ao veículo geograficamente mais próximo, calculando a distância euclidiana para cada uma das seis instâncias e evitando conflitos ou duplicações na atribuição de trabalho.

5.5 Variação do Atraso (Jitter)

Além da latência fixa, introduzimos uma variação aleatória no atraso, ou jitter, com uma variância de ± 500 ms. Isto fez com que muitos pacotes chegassem fora de ordem, como o pacote 5 chegar antes do 4. O sistema lidou com isto através da fila de reordenação no `udplink`. Ao receber o pacote 5, o recetor detetou a lacuna e reteve-o em buffer. Assim que o pacote 4 chegou, o sistema desbloqueou a fila e entregou ambos na ordem correta à aplicação. Nos casos extremos onde o atraso foi excessivo, o temporizador de segurança avançou a janela de receção para evitar bloquear a comunicação indefinidamente, registando o salto nos logs.

6. UTILIZAÇÃO DE INTELIGÊNCIA ARTIFICIAL

6.1.1 Aprendizagem da Linguagem e Produtividade

Como a linguagem Go possui paradigmas específicos, utilizámos a IA como um suporte à aprendizagem, o que nos permitiu compreender padrões idiomáticos e resolver dúvidas de sintaxe mais rapidamente, sem nunca substituir a compreensão lógica dos algoritmos. No quotidiano, o uso de funcionalidades de autocomplete ajudou a reduzir o tempo gasto em código repetitivo. No entanto, é fundamental sublinhar que o desenho e a lógica central dos protocolos `MissionLink` e `TelemetryStream` foram inteiramente concebidos e validados pelos membros do grupo.

6.1.2 Implementação do Ground Control e CLI da Nave-Mãe (Frontend)

A utilização de IA foi mais expressiva no desenvolvimento do Ground Control e da Interface de Linha de Comando da Nave-Mãe. Visto que o foco desta cadeira é a comunicação de dados e não o design de interfaces ou desenvolvimento web, optámos por recorrer à IA para gerar a base destas camadas de interação. Esta decisão permitiu-nos dispor de ferramentas de visualização funcionais sem desviar o esforço do desenvolvimento dos protocolos da rede, que constituem a nossa prioridade.

6.1.3 Scripts de Compilação e Execução

Adicionalmente, recorreu-se à IA para a criação de scripts de suporte essenciais ao fluxo de trabalho, nomeadamente o `Makefile` para a compilação automatizada dos binários do projeto e o script `run_simulation.py` para a orquestração do arranque da topologia e dos nós no emulador CORE. Esta automação permitiu agilizar significativamente o ciclo de desenvolvimento e testes, abstraindo a complexidade de configurar o ambiente e iniciar múltiplos processos manualmente a cada execução.

6.1.4 Elaboração do Relatório

Por fim, utilizamos ferramentas de IA para apoiar a revisão deste relatório, auxiliando na estruturação e na clareza da escrita. Ressalvamos, contudo, que a IA funcionou apenas como uma ferramenta de apoio editorial e nunca como autora. Todo o conteúdo, a análise crítica dos resultados e as conclusões aqui apresentadas são da inteira responsabilidade do grupo.

7. ANÁLISE CRÍTICA E CONCLUSÃO

Este projeto permitiu-nos simular e validar uma arquitetura de comunicações espaciais, enfrentando desafios reais como o transporte de dados instável e a tolerância a falhas. A solução final cumpre o que prometia e prova que a abordagem híbrida funciona: usar um protocolo UDP fiável (MissionLink) para o que é crítico e um fluxo TCP (TelemetryStream) para a monitorização revelou-se uma estratégia eficaz para gerir operações em redes difíceis.

7.1 Robustez

A robustez foi o ponto mais forte desta implementação. Isso deveu-se a escolhas de arquitetura que colocaram a segurança da memória e a resistência da rede em primeiro lugar.

7.1.1 Resiliência da Camada de Transporte

A implementação do udplink provou ser altamente resiliente. A estratégia de Retransmissão Seletiva (ARQ) combinada com Forward Error Correction (FEC) permitiu ao sistema recuperar de taxas de perda de pacotes superiores a 25% sem degradar a continuidade da missão. O uso de FEC reduziu a latência efetiva ao permitir a reconstrução de mensagens sem a necessidade de retransmissões custosas (em tempo) num ambiente de RTT elevado.

7.1.2 Integridade e Concorrência

A abolição de bloqueios globais em favor de primitivas de sincronização granulares eliminou condições de corrida (race conditions). O sistema manteve-se estável sob carga elevada, garantindo a consistência do estado global na Nave-Mãe mesmo durante o acesso simultâneo pela API de Observação e pelos handlers de rede.

7.1.3 Recuperação de Falhas

A arquitetura demonstrou "capacidade de cura". A utilização de Connection IDs para gestão de sessões, aliada aos mecanismos de watchdog (StaleCheck), assegurou que o sistema nunca permanecia num estado de bloqueio indefinido, recuperando automaticamente a sincronização dos Rovers após falhas críticas de software ou reinícios de rede.

7.2 Limitações e Melhorias Futuras

Apesar da eficácia da solução atual, a análise do desempenho e da arquitetura revela limitações inerentes ao escopo do projeto, que abrem caminho para otimizações significativas numa perspetiva de evolução para um ambiente de produção.

7.2.1 Limitações

O mecanismo atual de confirmação "um-para-um" (um ACK por cada fragmento recebido) no MissionLink gera um tráfego de controlo excessivo, desperdiçando largura de banda e ciclos de CPU.

O sistema transmite dados em texto claro, carecendo de mecanismos de encriptação e autenticação mútua, o que expõe a rede a riscos.

A centralização total na Nave-Mãe significa que uma falha neste nó paralisa a coordenação de toda a frota.

7.2.2 Melhorias Futuras

Para mitigar estas limitações e maximizar o throughput, propõem-se as seguintes evoluções técnicas:

Confirmações Agregadas (Cumulative/Selective ACKs): Substituir o envio de ACKs individuais por um mecanismo de SACK (Selective Acknowledgement). O recetor acumularia o estado de receção de múltiplos fragmentos (utilizando uma máscara de bits) e enviaria um único pacote de controlo para confirmar dezenas de fragmentos simultaneamente (e.g., "Confirmo fragmentos 1 a 10 e 15 a 20"). Isto reduziria drasticamente o overhead da rede.

Evoluir do backoff exponencial simples para algoritmos de janela deslizante baseados no atraso e largura de banda (como BBR ou Cubic adaptados a UDP), ajustando proativamente a taxa de envio para evitar a saturação do canal.

Codificação Delta: Implementar Delta Encoding na transmissão de telemetria. Em vez de enviar o estado completo do Rover a cada 100ms, enviar apenas os campos que sofreram alterações significativas desde o último envio, poupando largura de banda em variáveis estáveis (como a temperatura ou bateria).

7.3 Conclusão

Em suma, este trabalho permitiu consolidar conhecimentos fundamentais sobre a pilha protocolar e sistemas distribuídos. A solução desenvolvida não só resolve o problema proposto com robustez, como estabelece uma base modular sólida para a integração futura de otimizações.

8. APÊNDICE

8.1 Guia de Utilização

Este projeto implementa uma simulação de comunicação espacial entre uma Nave-Mãe (Mothership) e vários Rovers, monitorizados por um Ground Control (interface web). O sistema utiliza protocolos personalizados sobre TCP (TelemetryStream) e UDP (MissionLink).

8.1.1 Pré-requisitos

- Antes de iniciar, garanta que tem as seguintes ferramentas instaladas:
- Go (Golang): Versão 1.24 ou superior.
- Node.js & npm: Necessário para compilar a interface gráfica (Ground Control).
- CORE Emulator: Necessário apenas para a simulação de rede realista (Ambiente VM/Linux).
- Make: Para automação da compilação.
- Python 3: Para execução dos scripts de automação do cenário CORE.

8.1.2 Compilação (Build)

O projeto utiliza um Makefile para gerir a compilação de todos os componentes (backend e frontend).

Para limpar artefactos antigos e compilar os binários da Mothership, Rover e Ground Control (incluindo o frontend React):

```
BASH: make all
```

Compilar Componentes Individualmente

- Apenas Mothership: `make mothership`
- Apenas Rover: `make rover`
- Apenas Ground Control (Go + React): `make groundcontrol`

Os binários compilados serão colocados na pasta `bin/`.

8.1.3 Ajuste de Configurações

O sistema pode ser configurado de duas formas: via Flags de linha de comando (runtime) ou alterando Constantes no código (compile-time).

A. Configurações de Runtime (Flags)

Ao executar os binários manualmente, pode alterar endereços e portas:

Mothership:

- -ts-addr: Endereço do Telemetry Stream (TCP). Padrão: :9001
- -ml-addr: Endereço do Mission Link (UDP). Padrão: :9002
- -api-addr: Endereço da API HTTP. Padrão: :9003

Rover:

- -id: ID único do Rover.
- -m-ts-addr: Endereço TCP da Mothership.
- -m-ml-addr: Endereço UDP da Mothership.
- -r-ml-addr: Endereço UDP local do Rover

Ground Control:

- -port: Porta para servir o website. Padrão: :3000.
- -api-addr: URL da API da Mothership. Padrão: 10.0.0.21:8003

B. Configurações Internas (Código)

Para ajustes profundos no comportamento dos protocolos ou simulação, edite os seguintes ficheiros (ou opte por instanciar configurações sob medida):

Protocolo UDP (MissionLink) (pkg/transport/udplink/config.go):

- Timeouts: Define prazos de leitura/escrita (Read, Write), tempo de vida de pacotes fragmentados (RecvTTL) e tempo máximo de espera por pacotes fora de ordem (InOrder).
- Retransmission: Controla o número máximo de tentativas (MaxRetries), o tempo de espera inicial (InitialBackoff) e o multiplicador exponencial (BackoffMultiplier).
- FEC (Forward Error Correction): Define o MTU, o mínimo de fragmentos de dados e o rácio de redundância (ParityShardRatio) para reconstrução de pacotes perdidos sem retransmissão.
- Workers: Limites de concorrência (MaxRecvWorkers, MaxDeliveryWorkers).
- DefaultLoopTick: Frequência dos loops de verificação de retransmissão e limpeza.

Protocolo TCP (TelemetryStream) - (pkg/transport/tcpstream/config.go):

- DefaultClientTimeout: Timeouts de conexão (Dial) e envio (Write).
- DefaultReconnect: Estratégia de reconexão automática, incluindo InitialBackoff, MaxBackoff e BackoffMultiplier.
- DefaultServerTimeout: Prazos para aceitar conexões (Listen) e ler dados (Read), permitindo tolerar silêncios momentâneos na rede.

Frequência dos ciclos do Rover (cmd/rover/main.go):

- telemetryUpdateFrequency: Frequência com que o Rover envia dados de telemetria via TCP (padrão: 100ms).
- missionRequestFrequency: Frequência com que o Rover, estando livre, solicita novas missões via UDP (padrão: 5s).

Lógica da Mothership (cmd/mothership/main.go):

- staleMission: Número de ciclos de atualização falhados antes de considerar uma missão como "Unknown".
- staleRover: Tempo de silêncio (sem telemetria) antes de marcar um rover como "Unknown".

8.1.4 Execução do Projeto

Existem dois modos principais de execução: Localhost (desenvolvimento) e CORE Emulator (simulação de rede).

A. Localhost (Sem emulação de rede)

Este modo executa todos os componentes na máquina local (127.0.0.1) sem emulação de rede. É ideal para testar a lógica da aplicação, a interface gráfica e a comunicação básica sem a complexidade de perdas de pacotes ou latência.

Recomenda-se iniciar os componentes na seguinte ordem para garantir que as conexões TCP são estabelecidas corretamente:

1. Iniciar a Mothership: Execute o script auxiliar que levanta a mothership nas portas locais 9001-9003.

```
BASH: ./scripts/mothership.sh
```

2. Iniciar o Ground Control: Este script inicia o servidor web e abre o browser (Firefox) automaticamente na interface de controlo.

```
BASH: ./scripts/groundcontrol.sh
```

3. Iniciar a Frota de Rovers: Este script inicia 6 rovers, ligando-os à mothership local (127.0.0.1).

```
BASH: ./scripts/rover.sh
```

B. CORE Emulator (Com topologia de rede)

Este é o cenário principal do trabalho prático, onde a topologia de rede (satélites, perdas, latência) é aplicada.

Opção 1: Execução Automática (Script) Para simplificar o processo, existe um script em Python que orquestra a simulação. Este script é válido apenas para a topologia fornecida no projeto.

1. Executar o Script: Num terminal, execute o seguinte comando (requer privilégios de root para interagir com o CORE Daemon):

```
BASH: sudo python3 scripts/run_simulation.py
```

2. Fluxo de Execução:

- O script abrirá automaticamente o CORE GUI.
- No terminal, serão apresentadas instruções para carregar o ficheiro coreemu/topology.xml na interface gráfica e pressionar o botão Start.
- Após o arranque da emulação, pressione Enter no terminal para que o script inicie os binários (mothership, rover, groundcontrol) dentro dos respetivos nós virtuais.

Nota: Para análise de tráfego, pode utilizar o script alternativo scripts/run-sim-with-wireshark.py, que lança automaticamente o Wireshark no nó rover1.

Opção 2: Execução Manual (Acesso direto aos Nós) Caso pretenda depurar um componente específico ou correr o sistema passo-a-passo, pode aceder individualmente a cada nó virtual e executar os binários manualmente.

Para preparar o ambiente, comece por iniciar o core-daemon (como root/sudo) e a interface gráfica do emulador executando o comando core-gui. De seguida, carregue o ficheiro de topologia localizado em coreemu/topology.xml e inicie a simulação.

Para aceder e executar os componentes, clique com o botão direito no nó desejado na grelha do CORE e selecione Shell (ou CSH/Bash). Execute os comandos abaixo dentro das janelas de terminal que se abrirem.

Nó mothership (Servidor Central):

BASH:

```
cd /home/space-mission # Ajuste o caminho conforme necessário
./bin/mothership/mothership \
    -ts-addr=:9001 -ml-addr=:9002 -api-addr=:9003 \
```

```
load assets/missions/file1.txt
```

Nó ground (Interface de Controlo):

BASH:

```
cd /home/space-mission
./bin/groundcontrol/groundcontrol \
    -port=:3000 -api-addr=10.0.0.21:9003 \
    -dist=./bin/groundcontrol/dist
```

Nós rover1, etc. (Clientes):

BASH:

```
cd /home/space-mission
./bin/rover/rover \
    -id=1 -m-ts-addr=10.0.1.20:9001 \
    -m-ml-addr=10.0.1.20:9002 -r-ml-addr=:9000
```

8.1.5 Utilização da Interface (Ground Control)

Aceda à interface via browser (geralmente <http://localhost:3000>).

Funcionalidades:

1. Dashboard Principal (Grid Map):
 - Visualização em tempo real da posição dos Rovers.
 - Visualização das áreas de missão (Círculos ou Retângulos).
2. Painel Esquerdo (Rover Status & Available Fleet):
 - Lista todos os rovers detetados e a sua telemetria.
 - Permite filtrar por estado (Idle, On Mission, Error, Unknown).
3. Painel Direito (Mission Log & Control):
 - Lista as missões e o seu progresso (barra de percentagem).
 - Permite enviar novas missões para a Mothership.

8.1.6 CLI da Mothership

A Mothership possui uma interface de linha de comandos (CLI) interativa que corre no terminal onde o binário foi iniciado.

- `help`: Lista comandos disponíveis.

- `add <id> <task> ...`: Adiciona uma missão manualmente.
- `load <filename>`: Carrega missões a partir de um ficheiro de texto (ex: `assets/missions/file1.txt`).

8.1.7 Resolução de Problemas

Os icons/imagens da topologia ...

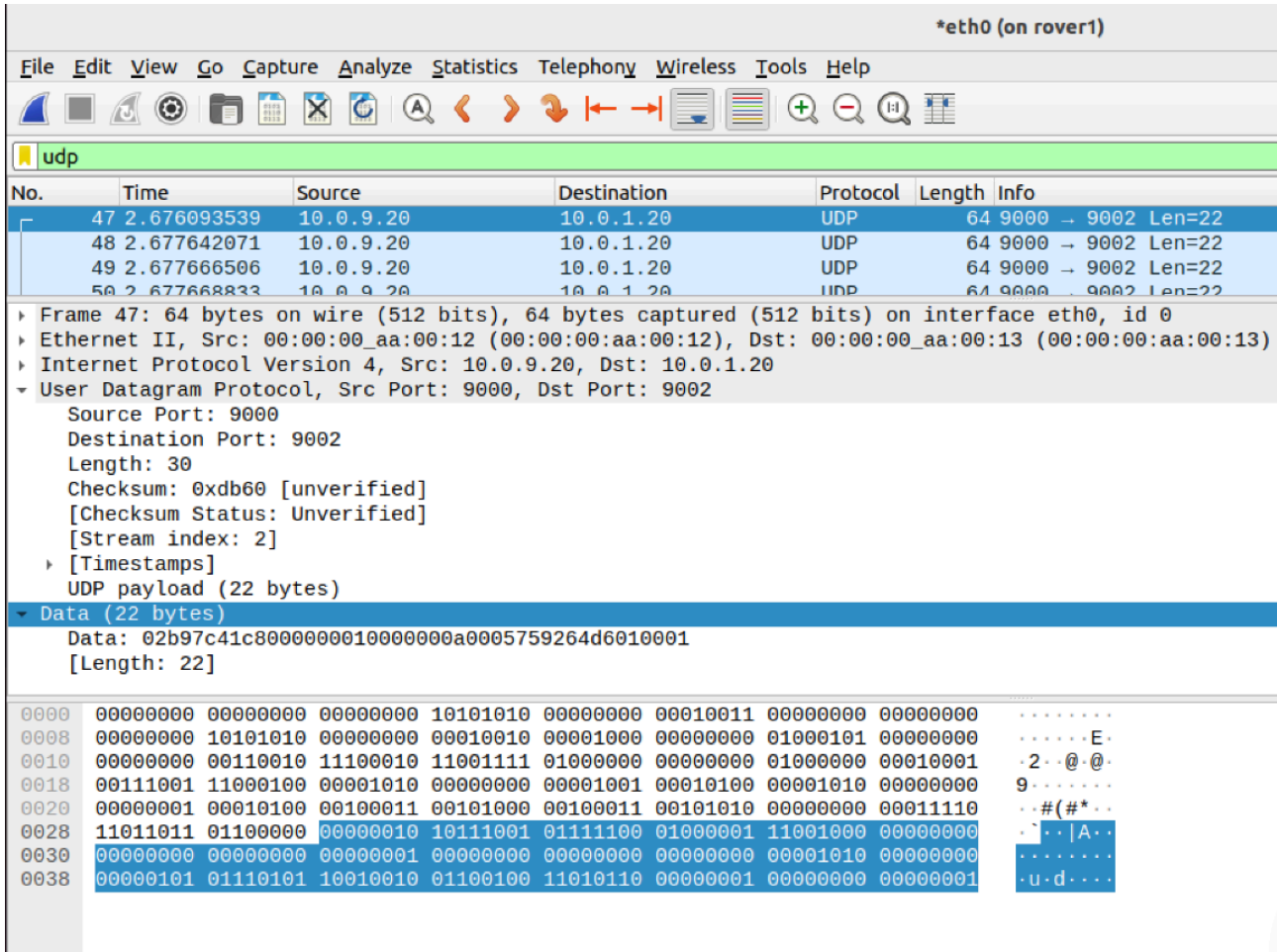
Cada componente gera ficheiros de log detalhados na pasta de execução (ex: `mothership.log`, `rover_1.log`, `mission_link_1.log`). Verifique estes ficheiros se houver falhas de comunicação.

Se obtiver erros de "address already in use", certifique-se de que não tem instâncias antigas a correr (`killall mothership rover groundcontrol`).

Se a interface não conectar, verifique se o endereço da API passado ao Ground Control (`-api-addr`) corresponde ao IP onde a Mothership está a correr (Localhost vs IP da rede CORE 10.0.0.21).

8.2 Anexos

8.2.1 FIGURA 1



8.2.2 FIGURA 2

*eth0 (on rover1)							
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help							
udp							
No.	Time	Source	Destination	Protocol	Length	Info	
47	2.676093539	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
48	2.677642071	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
49	2.677666506	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
50	2.677668833	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
51	2.677669969	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
52	2.677670984	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
53	2.677671991	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
54	2.677672951	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
55	2.679642658	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
56	2.679648683	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
57	2.679649786	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
58	2.679650848	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
59	2.679651925	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
60	2.679653063	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
61	2.679654097	10.0.9.20	10.0.1.20	UDP	64	9000 → 9002	Len=22
65	2.820093031	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
66	2.825411299	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
67	2.827395613	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
68	2.829127741	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
69	2.829129837	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
70	2.829131167	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
71	2.831782039	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
72	2.831914984	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
73	2.831916248	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
74	2.836022420	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
75	2.836024344	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
76	2.836026002	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
77	2.836027633	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
78	2.842791360	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19
79	2.842793076	10.0.1.20	10.0.9.20	UDP	61	9002 → 9000	Len=19

8.2.3 FIGURA 3

[illegible]